



Ağ Tabanlı Paralel Dağıtım Sistemleri

Final Projesi Kapsamlı Raporu

Yasin Turan Öcal

21290318

Proje GitHub Deposu

<https://github.com/2run66/ag-tabanli-final>

Proje Anlatım Videoları (Google Drive)

[Videoları İzlemek İçin Tıklayın](#)

Ağ Tabanlı Paralel Dağıtım Sistemleri - Kapsamlı Proje Raporu

Proje Başlıkları:

- Veritabanı Yedekleme ve Felaketten Kurtarma Planı (Proje 2)
- Veritabanı Güvenliği ve Erişim Kontrolü (Proje 3)
- Veri Ambarı ve ETL Süreçleri (Proje 5)

İçindekiler

- Bölüm 1: Proje 2 - Point-in-Time Recovery (PITR) ile Felaketten Kurtarma
 - 1.1 WAL Arşivleme Mimarisi ve Docker Yapılandırması
 - 1.2 Veritabanı Oluşturma ve İlk Veri Seti
 - 1.3 Fiziksel Tam Yedek (Base Backup) Süreci
 - 1.4 Felaket Senaryosu ve Kritik Zaman Damgası
 - 1.5 PITR Kurtarma Süreci ve Doğrulama
- Bölüm 2: Proje 3 - Veritabanı Güvenliği ve Erişim Kontrolü
 - 2.1 PGCrypto ile Veri Şifreleme
 - 2.2 Row Level Security (Satır Bazlı Güvenlik)
 - 2.3 Trigger Tabanlı Audit Logging (Denetim Kaydı)
 - 2.4 SQL Injection Saldırısı ve Savunması
- Bölüm 3: Proje 5 - Veri Ambarı ve ETL Süreçleri
 - 3.1 Kirli Veri Kaynağı (Extract Aşaması)
 - 3.2 Veri Dönüştürme (Transform Aşaması)
 - 3.3 Yıldız Şema Yükleme (Load Aşaması)
 - 3.4 Raporlama ve Sonuç Doğrulama

Bölüm 1: Proje 2 - Point-in-Time Recovery (PITR) ile Felaketten Kurtarma

Bir e-ticaret veritabanında felaketten kurtarma senaryosu uygulanarak, PostgreSQL'in WAL (Write-Ahead Logging) mekanizması ile belirli bir zaman noktasına (Point-in-Time) geri dönüş işleminin gerçekleştirilmesidir. Bu proje, Proje 7'deki otomasyon odaklı yedekleme stratejisinden farklı olarak **sıfır veri kaybı** hedefiyle fiziksel düzeyde kurtarma sürecini ele almaktadır.

1.1 WAL Arşivleme Mimarisi ve Docker Yapılandırması

PostgreSQL 16 aşağıdaki konfigürasyonla Docker üzerinde çalıştırılmıştır:

```
Docker Container
pg16_pitr (Port: 5433)

PostgreSQL 16
```

```

    wal_level = replica
    archive_mode = on
    archive_command = 'cp %p /mnt/...%f'

    Docker Volumes:
    pg_data_pitr      → /var/lib/pg/data
    pg_archive_pitr   → /mnt/archive
    pg_backup_pitr    → /mnt/backup

```

Volume Mimarisi:

Volume	Bağlanma Noktası	Amaç
pg_data_pitr	/var/lib/postgresql/data	Aktif veritabanı dosyaları
pg_archive_pitr	/mnt/archive	WAL arşiv dosyaları (sürekli yedek)
pg_backup_pitr	/mnt/backup	Fiziksel tam yedek (base backup)

WAL arşivleme aktif edildiğinde PostgreSQL, her tamamlanan WAL segmentini (**16 MB**) otomatik olarak **/mnt/archive** dizinine kopyalar. Bu sayede yedek alındıktan sonra yapılan tüm değişiklikler (INSERT, UPDATE, DELETE) kayıt altında tutulur.

1.2 Veritabanı Oluşturma ve İlk Veri Seti

e_ticaret_db veritabanında iki ilişkisel tablo oluşturulmuştur:

```

CREATE TABLE musteriler (
    id SERIAL PRIMARY KEY,
    ad VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    kayit_tarihi TIMESTAMP DEFAULT current_timestamp
);
CREATE TABLE siparisler (
    id SERIAL PRIMARY KEY,
   musteri_id INT REFERENCES musteriler(id),
    toplam_tutar DECIMAL(10, 2),
    durum VARCHAR(50),
    siparis_tarihi TIMESTAMP DEFAULT current_timestamp
);

```

Başlangıç Verileri:

id	musteri_id	toplam_tutar	durum	siparis_tarihi
1	1	1500.00	Tamamlandı	2026-07-16 19:24:47.27218
2	2	450.50	Hazırlanıyor	2026-07-16 19:24:47.27218

```
3 | 1 | 100.00 | İptal | 2026-07-16 19:24:47.27218  
(3 rows)
```

1.3 Fiziksel Tam Yedek (Base Backup) Süreci

`pg_basebackup` komutu ile veritabanının fiziksel düzeyde birebir kopyası alınmıştır:

```
pg_basebackup -U admin -D /mnt/backup/base_backup -F p -X stream
```

Parametre	Açıklama
<code>-U admin</code>	Yetkili veritabanı kullanıcısı
<code>-D /mnt/backup/base_backup</code>	Yedeğin yazılacağı hedef dizin
<code>-F p</code>	Format: Plain (klasör yapısı, veri dizininin birebir kopyası)
<code>-X stream</code>	Yedekleme sırasında oluşan WAL kayıtlarını da eş zamanlı olarak yedeğe dahil eder

Bu yedek, veritabanının o anki fiziksel durumunun (data dosyaları, WAL konumu dahil) tam bir kopyasıdır.

1.4 Felaket Senaryosu ve Kritik Zaman Damgası

Yedek alındıktan sonra veritabanına yeni bir sipariş eklenmiş ve ardından kasıtlı olarak felaket senaryosu uygulanmıştır:

1. Yeni Veri Ekleme (Yedek Sonrası):

```
INSERT INTO siparisler (musteri_id, toplam_tutar, durum) VALUES (3,  
5000.00, 'Kargoda');
```

2. Kritik Zaman Damgası Kaydetme:

```
SELECT to_char(current_timestamp + interval '2 seconds', 'YYYY-MM-DD  
HH24:MI:SS');  
-- Sonuç: '2026-07-16 19:26:50'
```

Bu zaman damgası, PITR kurtarma hedefimizdir. Veritabanını tam olarak bu ana kadar geri getireceğiz.

3. WAL Segment Tetikleme:

```
SELECT pg_switch_wal();
```

Yeni eklenen verinin WAL arşivine yazılmasını garanti eder.

4. Felaket — Tablo Silme:

```
DROP TABLE siparisler CASCADE;
```

Siparişler tablosu ve tüm veriler yok edilmiştir. `\dt siparisler` sorgusu "Did not find any relation" döner.

1.5 PITR Kurtarma Süreci ve Doğrulama

Kurtarma 4 aşamada gerçekleştirilmiştir:

Aşama 1 — Container Durdurma:

```
docker compose down
```

Aşama 2 — Veri Dizinini Temizleyip Base Backup'ı Geri Yükleme:

```
# Aktif veri dizinini tamamen silip, fiziksel yedeği kopyalama
docker run --rm \
  -v proje2_pg_data_pitr:/data \
  -v proje2_pg_backup_pitr:/backup \
  busybox sh -c "rm -rf /data/* && cp -a /backup/base_backup/* /data/"
```

Aşama 3 — Recovery Yapılandırması:

```
docker run --rm -v proje2_pg_data_pitr:/data busybox sh -c \
  "touch /data/recovery.signal && \
  echo \"restore_command = 'cp /mnt/archive/%f %p'\" >> \
  /data/postgresql.auto.conf && \
  echo \"recovery_target_time = '2026-07-16 19:26:50'\" >> \
  /data/postgresql.auto.conf && \
  echo \"recovery_target_action = 'promote'\" >> \
  /data/postgresql.auto.conf && \
  chown -R 999:999 /data"
```

Parametre	Açıklama
<code>recovery.signal</code>	PostgreSQL'e kurtarma modunda başlamasını söyleyen tetikleyici dosya
<code>restore_command</code>	Arşivlenmiş WAL dosyalarını kurtarma sırasında nereden kopyalayacağını belirtir

Parametre	Açıklama
<code>recovery_target_time</code>	Kurtarma işleminin durması gereken kesin zaman noktası
<code>recovery_target_action = 'promote'</code>	Hedef zamana ulaşılmca veritabanını otomatik olarak normal (yazılabilir) moda geçirir

Aşama 4 — Kurtarma ve Doğrulama:

```
docker compose up -d
# 15 saniye bekleme (WAL replay süreci)
```

Kurtarma Sonucu:

```
id | müşteri_id | toplam_tutar | durum | siparis_tarihi
---+-----+-----+-----+-----
--
1 | 1 | 1500.00 | Tamamlandı | 2026-07-16 19:24:47.27218
2 | 2 | 450.50 | Hazırlanıyor | 2026-07-16 19:24:47.27218
3 | 1 | 100.00 | İptal | 2026-07-16 19:24:47.27218
4 | 3 | 5000.00 | Kargoda | 2026-07-16
19:26:48.099516
(4 rows)
```

Sonuç: Yedek sonrası eklenen 4. satır (id=4, **Kargoda**) dahil olmak üzere **tüm veriler %100 kurtarılmıştır**. **DROP TABLE** işlemi geri alınmış, veri kaybı sıfırdır.

Bölüm 2: Proje 3 - Veritabanı Güvenliği ve Erişim Kontrolü

PostgreSQL üzerinde ileri düzey güvenlik mekanizmalarının (şifreleme, satır bazlı erişim kontrolü, denetim kaydı ve SQL injection koruması) uygulanmasıdır.

2.1 PGCrypto ile Veri Şifreleme

pgcrypto eklentisi kullanılarak hassas finansal veriler (kredi kartı numaraları) veritabanı seviyesinde şifrelenmiştir.

Şifreleme İşlemi:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Kredi kartı PGP simetrik şifreleme ile saklanıyor
INSERT INTO finans_verileri (kullanici_id, kredi_karti, bakiye, departman)
VALUES
(1, PGP_SYM_ENCRYPT('4545-1234-5678-9999', 'SüperGizliAnahtar'), 50000.00,
'satis'),
(2, PGP_SYM_ENCRYPT('5555-4444-3333-2222', 'SüperGizliAnahtar'),
150000.00, 'yonetici');
```

Şifreli Veri Görünümü (SELECT Sonucu):

id	kredi_karti	bakiye
1	\xc30d0404030289a1e82f... (şifreli)	50000.00
2	\xc30d04040302e8f3bc4a... (şifreli)	150000.00

Veritabanına erişim sağlansa bile kredi kartı numaraları okunamaz durumdadır.

Şifre Çözme (Doğru Anahtar ile):

```
SELECT id,
       PGP_SYM_DECRYPT(kredi_karti::bytea, 'SüperGizliAnahtar') AS
acik_kredi_karti,
       bakiye
FROM finans_verileri;
```

id	acik_kredi_karti	bakiye
1	4545-1234-5678-9999	50000.00
2	5555-4444-3333-2222	150000.00

Şifreleme Teknikleri:

Fonksiyon	Amaç	Kullanım
<code>PGP_SYM_ENCRYPT()</code>	Simetrik PGP şifreleme (AES)	Kredi kartı, kişisel veriler
<code>PGP_SYM_DECRYPT()</code>	Simetrik PGP şifre çözme	Yetkili kullanıcı sorguları
<code>crypt() + gen_salt('bf')</code>	Bcrypt ile parola hashleme	Kullanıcı şifreleri

2.2 Row Level Security (Satır Bazlı Güvenlik)

RLS politikası ile her kullanıcı sadece kendi departmanına ait verileri görebilmektedir:

Politika Tanımı:

```
ALTER TABLE finans_verileri ENABLE ROW LEVEL SECURITY;

-- Satış personeli sadece 'satis' departmanındaki verileri görebilir
CREATE POLICY satis_gorme_politikasi ON finans_verileri
  FOR SELECT TO satis_rolu
  USING (departman = 'satis');

-- Yönetici her şeyi görebilir
CREATE POLICY yoneticisi_politikasi ON finans_verileri
  FOR ALL TO yoneticisi_rolu
  USING (true);
```

Test Sonucu (satis_usr kullanıcısı ile bağlanma):

```
-- satis_usr olarak: sadece departman = 'satis' olan satırlar gelir
id | kullanıcı_id | kredi_karti | bakiye | departman
---+-----+-----+-----+-----
  1 |          1 | \xc30d0404... (şfr.) | 50000.00 | satis
(1 row)

-- 'yoneticisi' departmanındaki 2. satır RLS tarafından GİZLENMİŞTİR
```

2.3 Trigger Tabanlı Audit Logging (Denetim Kaydı)

Hassas tablolardaki her UPDATE ve DELETE işlemi otomatik olarak `audit_log` tablosuna kaydedilmektedir:

Trigger Fonksiyonu:

```
CREATE OR REPLACE FUNCTION audit_trigger_func() RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'UPDATE' THEN
    INSERT INTO audit_log (tablo_adi, islem_tipi, eski_deger,
      yeni_deger)
```

```

VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD)::jsonb,
row_to_json(NEW)::jsonb);
RETURN NEW;
ELSIF TG_OP = 'DELETE' THEN
INSERT INTO audit_log (tablo_adi, islem_tipi, eski_deger)
VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD)::jsonb);
RETURN OLD;
END IF;
END;
$$ LANGUAGE plpgsql;

```

Test — Bakiye Güncelleme ve Audit Log:

```
UPDATE finans_verileri SET bakiye = 75000.00 WHERE id = 1;
```

tablo_adi	islem_tipi	eski_bakiye	yeni_bakiye	islem_yapan
finans_verileri	UPDATE	50000.00	75000.00	admin

Eski ve yeni değerler JSONB formatında saklanarak **tam izlenebilirlik** sağlanmıştır.

2.4 SQL Injection Saldırısı ve Savunması

Dinamik SQL kullanan zafiyetli bir fonksiyon ile SQL Injection saldırısının nasıl gerçekleştiği ve parametrik sorgu ile nasıl engellendiği gösterilmiştir.

Zafiyetli Fonksiyon (Concatenation ile):

```

CREATE OR REPLACE FUNCTION bakiye_getir_zayif(kullanici_isim VARCHAR)
RETURNS TABLE(bakiye DECIMAL) AS $$
DECLARE sorgu TEXT;
BEGIN
-- TEHLİKELİ: Parametre doğrudan SQL stringine ekleniyor
sorgu := 'SELECT f.bakiye FROM finans_verileri f JOIN kullanicilar k
ON k.id = f.kullanici_id WHERE k.kullanici_adi = '''
|| kullanici_isim || ''';
RETURN QUERY EXECUTE sorgu;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Normal Kullanım:

```

SELECT * FROM bakiye_getir_zayif('ahmet_satis');
-- Sonuç: 50000.00 (Sadece Ahmet'in bakiyesi)

```

SQL Injection Saldırısı:

```
SELECT * FROM bakiye_getir_zayif('ahmet_satis' OR '1'='1');
```

```

bakiye
-----
50000.00
150000.00
(2 rows)
-- TÜM kullanıcıların bakiyeleri sızdırıldı!

```

Saldırganın gönderdiği ' OR '1'='1 ifadesi WHERE koşulunu bypass ederek tüm satırları döndürmüştür.

Güvenli Fonksiyon (Parametrik Sorgu — USING ile):

```

CREATE OR REPLACE FUNCTION bakiye_getir_guvenli(kullanici_isim VARCHAR)
RETURNS TABLE(bakiye DECIMAL) AS $$
BEGIN
    -- GÜVENLİ: EXECUTE ... USING ile parametre string olarak işlenir
    RETURN QUERY EXECUTE
        'SELECT f.bakiye FROM finans_verileri f JOIN kullanicilar k
        ON k.id = f.kullanici_id WHERE k.kullanici_adi = $1'
    USING kullanici_isim;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Aynı Saldırı Güvenli Fonksiyona Yapılınca:

```

SELECT * FROM bakiye_getir_guvenli('ahmet_satis' OR '1'='1');
-- Sonuç: 0 rows (BOŞ – saldırı engellendi)

```

Yöntem	Güvenlik	Açıklama
String Concatenation ()	✗ Zafiyetli	Kullanıcı girdisi doğrudan SQL'e eklenir
Parametrik Sorgu (USING)	✓ Güvenli	Girdi bir parametre olarak işlenir, SQL yapısını değiştiremez

Bölüm 3: Proje 5 - Veri Ambarı ve ETL Süreçleri

Kirli ve düzensiz verinin Extract (Çıkarma), Transform (Dönüştürme) ve Load (Yükleme) süreçleriyle temizlenip Yıldız Şema (Star Schema) mimarisine aktarılmasıdır. RegEx, JSON ayrıştırma, pencere fonksiyonları ve stored procedure teknikleri kullanılmıştır.

3.1 Kirli Veri Kaynağı (Extract Aşaması)

Dış sistemlerden gelen, kasıtlı olarak kirletilmiş ham veri tablosu oluşturulmuştur:

```
id |   müşteri_tam_isim   |   iletişim_bilgisi   |
| siparis_tarihi | satis_tutari |   urun_detay
-----+-----+-----
1 | ALİ yılmaz          | {"email": "ali@mail.com", "telefon": "555-123-4567"} | 2023-10-12 | 1250.50 TL | Elektronik - Laptop
2 | ayşe Kaya           | {"email": "ayse.kaya@mail", "telefon": "0532..."} | 15/11/2023 | $200      | Giyim - Kazak
3 | Mehmet DEMİR        | {"email": "mehmet@mail.com"} | 2023/12/01 | NULL      | Elektronik - Telefon
4 | ALİ yılmaz          | (KOPYA KAYIT - id=1 ile aynı) | 2023-10-12 | 1250.50 TL | Elektronik - Laptop
5 | Zeynep Çelik        | Sadece text veri var, json değil. tel: 0544-333... | InvalidDate | 750 TL    | Kozmetik - Parfüm
```

Tespit Edilen Veri Kalitesi Sorunları:

#	Sorun	Örnek
1	Gereksiz boşluklar ve büyük/küçük harf tutarsızlığı	" ALİ yılmaz "
2	İletişim bilgisi bazen JSON, bazen düz metin	"Sadece text veri..." vs {"email":...}
3	Farklı tarih formatları	2023-10-12, 15/11/2023, 2023/12/01, InvalidDate
4	Para birimi simgeleri ile karışık tutar	"1250.50 TL", "\$200", "NULL"
5	Mükerrer (duplicate) kayıtlar	id=1 ve id=4 aynı veri

3.2 Veri Dönüştürme (Transform Aşaması)

`etl_surecini_baslat()` Stored Procedure'ü ile tüm dönüştürme kuralları tek bir atomik işlemde çalıştırılmaktadır:

3.2.1 Deduplication (Mükerrer Kayıt Temizliği):

```
CREATE TEMP TABLE ham_veri_tekil AS
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY musteri_tam_isim, siparis_tarihi, urun_detay
        ORDER BY id
    ) as satir_no
    FROM satislar_ham
) sub WHERE satir_no = 1;
```

`ROW_NUMBER()` pencere fonksiyonu ile aynı müşteri adı, tarih ve ürün detayına sahip kayıtlar gruplanır. Her grubun yalnızca ilk satırı (`satir_no = 1`) tutularak 5 kayıttan → 4 tekil kayıt elde edilir.

3.2.2 İsim Standardizasyonu (String Manipulation):

```
INITCAP(TRIM(SPLIT_PART(TRIM(musteri_tam_isim), ' ', 1))) AS ad,
UPPER(TRIM(SUBSTRING(TRIM(musteri_tam_isim) FROM POSITION(' ' IN
TRIM(musteri_tam_isim))))) AS soyad
```

Fonksiyon	İşlev	Örnek Dönüşüm
<code>TRIM()</code>	Baş/son boşlukları siler	" ALİ yılmaz " → "ALİ yılmaz"
<code>SPLIT_PART(, ' ', 1)</code>	Boşluğa göre böler, 1. parçayı alır	"ALİ yılmaz" → "ALİ"
<code>INITCAP()</code>	İlk harf büyük, kalanı küçük	"ALİ" → "Ali"
<code>UPPER()</code>	Tamamını büyük harf yapar	"yılmaz" → "YILMAZ"

3.2.3 JSON Parse ve RegEx ile İletişim Verisi Ayıklama:

```
-- E-posta:
CASE
    WHEN iletisim_bilgisi LIKE '{%' THEN iletisim_bilgisi::jsonb ->>
    'email'
    ELSE SUBSTRING(iletisim_bilgisi FROM '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+' )
END AS email

-- Telefon:
CASE
    WHEN iletisim_bilgisi LIKE '{%' THEN
    REGEXP_REPLACE(iletisim_bilgisi::jsonb ->> 'telefon', '\D', '', 'g')
    ELSE REGEXP_REPLACE(SUBSTRING(iletisim_bilgisi FROM '0[0-9-]{10,}'),
    '\D', '', 'g')
END AS telefon
```

Teknik	Kullanım Amacı
LIKE '{%}'	Metnin JSON olup olmadığını kontrol eder
::jsonb ->> 'email'	JSON nesnesinden belirli bir alanı çeker
SUBSTRING(... FROM 'regex')	Düz metinden RegEx deseniyle e-posta veya telefon ayıklar
REGEXP_REPLACE(_, '\D', '', 'g')	Rakam olmayan tüm karakterleri siler (telefon temizliği)

3.2.4 Tarih Formatı Standardizasyonu:

```

CASE
    WHEN siparis_tarihi ~ '^\\d{4}-\\d{2}-\\d{2}$' THEN CAST(siparis_tarihi
AS DATE)          -- YYYY-MM-DD
    WHEN siparis_tarihi ~ '^\\d{2}/\\d{2}/\\d{4}$' THEN
TO_DATE(siparis_tarihi, 'DD/MM/YYYY') -- DD/MM/YYYY
    WHEN siparis_tarihi ~ '^\\d{4}/\\d{2}/\\d{2}$' THEN
TO_DATE(siparis_tarihi, 'YYYY/MM/DD')  -- YYYY/MM/DD
    ELSE NULL -- 'InvalidDate' gibi geçersiz değerler
END AS tarih

```

3.2.5 Tutar ve Para Birimi Ayırıştırma:

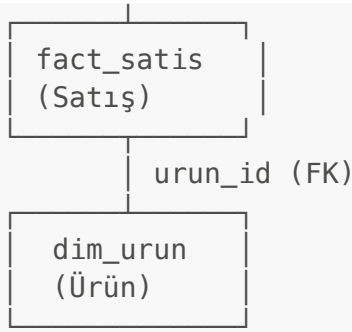
```
-- Tutar (rakamsal değer):
CASE
    WHEN satis_tutari = 'NULL' OR satis_tutari IS NULL THEN 0
    ELSE CAST(REGEXP_REPLACE(satis_tutari, '[^\d.]', '', 'g') AS DECIMAL)
END AS tutar

-- Para birimi (ayrı sütun):
CASE
    WHEN satis_tutari LIKE '%$%' THEN 'USD'
    WHEN UPPER(satis_tutari) LIKE '%TL%' THEN 'TRY'
    ELSE 'TRY'
END AS para_birimi
```

3.3 Yıldız Şema Yükleme (Load Aşaması)

Temizlenen veriler, veri ambarı modelinde üç tabloya yüklenmiştir:



**Boyut Tabloları (Dimension):**

dim_musteri (Temizlenmiş Müşteri Verisi):

musteri_id	ad	soyad	email	telefon
1	Ali	YILMAZ	ali@mail.com	5551234567
2	Ayşe	KAYA	ayse.kaya@mail	05329998877
3	Mehmet	DEMİR	mehmet@mail.com	
4	Zeynep	ÇELİK		05443332211

dim_urun (Ayrıştırılmış Ürün Verisi):

urun_id	kategori	urun_adi
1	Elektronik	Laptop
2	Giyim	Kazak
3	Elektronik	Telefon
4	Kozmetik	Parfüm

Gerçek Tablosu (Fact):

fact_satis:

satis_id	musteri_id	urun_id	tarih	tutar	para_birimi
1	1	1	2023-10-12	1250.50	TRY
2	2	2	2023-11-15	200.00	USD
3	3	3	2023-12-01	0.00	TRY
4	4	4		750.00	TRY

3.4 Raporlama ve Sonuç Doğrulama

Yıldız Şemadaki tabloları birleştiren **vw_satis_raporu** View'i ile nihai rapor:

```
CREATE VIEW vw_satis_raporu AS
SELECT f.tarih, m.ad || ' ' || m.soyad AS muster_i, m.telefon,
       u.kategori, u.urun_adi, f.tutar, f.para_birimi
FROM fact_satis f
JOIN dim_musteri m ON m.musteri_id = f.musteri_id
JOIN dim_urun u ON u.urun_id = f.urun_id;
```

Rapor Çıktısı:

```
   tarih   |   muster_i   |   telefon   | kategori   | urun_adi |
tutar   | para_birimi
-----+-----+-----+-----+-----+-----
---+-----
 2023-10-12 | Ali YILMAZ   | 5551234567  | Elektronik | Laptop   |
1250.50 | TRY
 2023-11-15 | Ayşe KAYA    | 05329998877 | Giyim      | Kazak    |
200.00  | USD
 2023-12-01 | Mehmet DEMİR |              | Elektronik | Telefon   |
0.00    | TRY
              | Zeynep ÇELİK | 05443332211 | Kozmetik   | Parfüm    |
750.00  | TRY
```

ETL Sürecinde Kullanılan PostgreSQL Fonksiyonları Özeti:

Kategori	Fonksiyonlar
Metin İşleme	TRIM, INITCAP, UPPER, SPLIT_PART, SUBSTRING, POSITION
Düzenli İfadeler (Regex)	REGEXP_REPLACE, SUBSTRING ... FROM 'pattern', ~ operatörü
JSON İşleme	::jsonb, ->> operatörü, LIKE '%{' kontrolü
Pencere Fonksiyonları	ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)
Tip Dönüşümü	CAST(... AS DATE), CAST(... AS DECIMAL), TO_DATE()
Stored Procedure	CREATE OR REPLACE PROCEDURE ... CALL

Dosya Yapıları

```
proje2/
├── docker-compose.yml      # PG16 + WAL arşivleme (PITR için)
├── VIDEO_KOMUTLARI.sh      # Video çekimi için interaktif test scripti
├── sql/
│   └── 01_init.sql         # E-ticaret veritabanı tabloları ve örnek
veri
proje3/
├── docker-compose.yml      # PG16 güvenlik testi ortamı
```

```
|— VIDEO_KOMUTLARI.sh      # Video çekimi için interaktif test scripti
|— sql/
|   |— 01_advanced_security.sql # PGCrypto, RLS, Audit, SQL Injection

proje5/
|— docker-compose.yml      # PG16 ETL/veri ambarı ortamı
|— VIDEO_KOMUTLARI.sh      # Video çekimi için interaktif test scripti
|— sql/
|   |— 01_advanced_etl.sql   # Kirli veri, Star Schema, ETL Stored
Procedure
```